

Advancing Blockchain-based Federated Learning through Verifiable Off-chain Computations

Jonathan Heiss, Elias Grünewald, Stefan Tai
Information Systems Engineering
TU Berlin
Berlin, Germany
{jh, eg, st}@ise.tu-berlin.de

Nikolas Haimerl
Distributed Systems Group
TU Wien
Wien, Austria
s01452766@student.tuwien.ac.at

Stefan Schulte
Christian Doppler Laboratory
for Blockchain Technologies
for the Internet of Things, TU Hamburg
Hamburg, Germany
stefan.schulte@tuhh.de

Abstract—Federated learning may be subject to both global aggregation attacks and distributed poisoning attacks. Blockchain technology along with incentive and penalty mechanisms have been suggested to counter these. In this paper, we explore verifiable off-chain computations using zero-knowledge proofs as an alternative to incentive and penalty mechanisms in blockchain-based federated learning. In our solution, learning nodes, in addition to their computational duties, act as off-chain provers submitting proofs to attest computational correctness of parameters that can be verified on the blockchain. We demonstrate and evaluate our solution through a health monitoring use case and proof-of-concept implementation leveraging the ZoKrates language and tools for smart contract-based on-chain model management. Our research introduces verifiability of correctness of learning processes, thus advancing blockchain-based federated learning.

Index Terms—federated learning, blockchain, off-chain computations, verifiability, zero-knowledge proofs, zokrates

I. INTRODUCTION

Machine learning is these days seen as an indispensable feature in many different fields, for instance, smart cities [1] or smart healthcare [2]. Despite the potential benefits of machine learning, privacy concerns are very often named as a major hindrance for its wide-spread adoption. This is especially the case if personal data needs to be processed [3]. In conventional settings, machine learning is conducted in a centralized style, very often making use of cloud-based computational resources for the training of a machine learning model. This may lead to a large communication overhead, if data from a large number of distributed data sources, e.g., in the Internet of Things (IoT), needs to be provided to the cloud for the learning tasks.

In order to overcome these issues, *federated learning* offers a different approach to machine learning. Instead of the traditional centralized setting, federated learning distributes the model training among nodes, which are often located at the edge of the network and therefore close to the data sources [4], [5]. Hence, the training data does not have to be shared with any centralized node. Instead, only the learned model is shared with some aggregation entity. Apart from the apparent improvement with regard to data privacy due to the lack of sharing of raw training data, this also decreases the communication overhead significantly.

Because of its distributed nature, federated learning may however suffer from malicious contributors, i.e., distributed

learners who provide an inaccurate or even fake local model to the aggregator [6], [7]. Such distributed *poisoning attacks* may decrease the integrity of the resulting machine learning model severely. In addition, the aggregator may itself attack the integrity of the resulting model, if the aggregation process is not done in a transparent way. For instance, the aggregator could skew the resulting model by weighting some local models higher than others. This is also known as a *global aggregation attack*. Therefore, it is necessary to provide mechanisms that impede the occurrence or consequences of attacks both by the decentralized learners and the aggregators.

Several recent studies have proposed to apply blockchain technologies for this, i.e., to do the model aggregation on-chain [8]–[13], and to offer incentive and penalty mechanisms which provide a stimulus to the decentralized learners to deliver verified local models [14]–[17]. One common issue with these solutions are the occurring costs for verifying the local models: Especially if making use of a public blockchain, carrying out heavyweight computations in smart contracts may lead to (very) high gas costs [18], which diminishes the number of application scenarios for which blockchain-based federated learning is useful.

In this paper, we propose an alternative system architecture for trustworthy federated learning, leveraging smart contracts to aggregate global learning models, and zero-knowledge proofs (ZKPs) to make the computational correctness of the local learning processes verifiable on the blockchain. For this, we model and implement federated learning using blockchain technology along with verifiable off-chain computations (VOC). Our contributions can be summarized as follows:

- We propose and implement a system architecture for federated learning that leverages blockchain technology to provide decentralized, tamper-resistant, and globally verifiable management of global learning parameters and VOC to enable verifying the computational correctness of local learning processes on the blockchain.
- We instantiate this architecture in a scenario where sensitive data serve as inputs to the federated learning system illustrating the feasibility of our approach. Furthermore, we evaluate our findings with regard to learning performance and computational costs.

The remainder of this paper is structured as follows: First, we summarize preliminaries on federated learning, blockchains, and ZKPs in Section II. Afterwards, we introduce our general system design (Section III) and describe the system's implementation (Section IV). We present a specific showcase scenario from the field of smart healthcare in Section V and evaluate the presented scenario in Section VI. Finally, we assess the related work (Section VII) and conclude our paper (Section VIII). An extended version of this paper can be found here¹.

II. BACKGROUND

Our research combines the two previously separated concepts of federated learning and VOC.

A. Federated Learning

Federated learning provides an architectural approach to apply machine learning models to a distributed setting where many nodes jointly train a shared global model in an efficient and privacy-preserving manner [19].

A learning model typically consists of learning parameters, e.g., weights and biases. In Artificial Neural Networks (ANNs), the learning parameters are contained in the network's hidden layers and applied to the input data during forward propagation to determine the learning output, e.g., a prediction class. In ANNs for supervised learning, this learning output is compared to predetermined truth labels during backpropagation to determine the learning error. Based on that error, the learning parameters are re-calculated and then applied to the model's hidden layers for the next learning cycle [20].

In federated learning, learning parameters are computed as local models by distributed learning nodes on local data. Each participating node provides only its model to an aggregator that combines all received models into a global model which represents the collective learning insights of the learning network. This global model is then locally used by the learning nodes in the next training cycle. Thereby, each learning node benefits from the training executed at other nodes without revealing potentially confidential inputs [19], [21].

While federated learning has originally been proposed as centralized architecture [21] where the global model is managed by a single aggregator, decentralized architectures have also been proposed where the management of the global model is jointly executed by the learning nodes, e.g., [8]–[17].

B. Verifiable Off-chain Computations

Blockchain technology enables mutually distrusting stakeholders to jointly maintain state in a trustless manner, that means, no single party needs to be trusted, but instead the system design itself provides trust guarantees. It does this through both, a cryptographically secured, append-only data structure and an incentive-driven [22] or Byzantine [23] consensus protocol. While the design of blockchains enables tamper-resistance and public verifiability in distributed environments

and, thereby, represents a good fit for managing global parameters in federated learning settings, it also causes weak scalability and leaking privacy. One conceptual approach to mitigate these deficiencies are VOC.

In VOC [24], a computation that would originally be executed on-chain is outsourced to an arbitrary off-chain node. In addition to the computational result, the off-chain node generates a proof that attests to the computation's correctness and can be verified on the blockchain. Thereby, VOC enables on-chain verification of computations on confidential data without revealing it on-chain. Furthermore, it mitigates scalability limitations since an arbitrary large computation can be executed non-redundantly by a scalable off-chain node and only a fixed-size proof smaller than the computation itself is verified as part of the costly consensus protocol. Thereby, extending blockchain-based federated learning with VOC enables on-chain verifiability of off-chain learning processes.

As an enabler for VOC, *ZoKrates* [18] has been proposed, a technology that provides a language and toolbox for zkSNARKs-based VOC. zkSNARKs (zero-Knowledge Succinct Non-interactive Argument of Knowledge) represent a specific zero-knowledge protocol that distinguishes through non-interactivity and succinct proof sizes. *ZoKrates* hides the peculiarities of zkSNARKs and provides a convenient, developer-friendly means for implementing and deploying both, the off-chain proving program and the on-chain verification smart contract for the Ethereum blockchain [22]. In this paper, we leverage *ZoKrates* to implement the local learning models as off-chain provers.

III. SYSTEM DESIGN

Addressing prevalent security issues in federated learning systems exposed by the local model training and the global model management, we propose a system design that strives towards the following objectives:

- *Tamper-resistance*: No party can maliciously manipulate a learning model, i.e., neither the local nor the global model.
- *Global verifiability*: The computation of the local and the global model must be verifiable by all participating nodes.

To achieve these objectives, our system architecture has two distinguishing features. First, the global model is stored on the blockchain and managed by smart contracts. This enables public verifiability and tamper resistance through system guarantees of the underlying blockchain. Second, the local model is implemented as a realization of VOC as described in Section II-B. This extends public verifiability and tamper resistance towards computations executed off-chain on the local model while preserving the privacy guarantees of federated learning. We thereby address the initially stated problem of non-trusted learning nodes that weaken the global model's integrity by deliberately or accidentally corrupting local computations that return false inputs for the global model updates.

¹<https://arxiv.org/pdf/2206.11641.pdf>

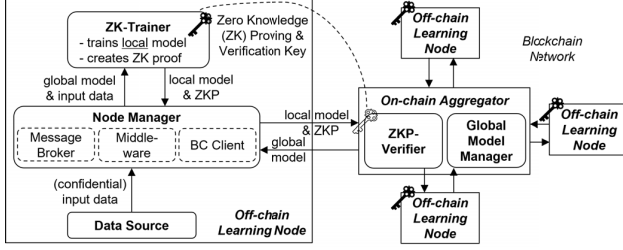


Fig. 1: Overview

A. System Overview

The system, as depicted in Figure 1, can be described from the two federated learning perspectives: the local perspective comprises components running on individual *off-chain learning nodes* and the global perspective considers the joint management of the global model through all learning nodes redundantly executing the *on-chain aggregator*.

1) *Off-chain Learning Node*: On the off-chain learning node, the *data source* creates the data that is input to the local training process and may contain confidential information.

The *node manager* handles the interaction between the data source, zk-trainer, and the on-chain aggregator. For this, it implements a *message broker* to accept messages from the data source which in turn are provided through the middleware to the zk-trainer. Furthermore, it implements a *blockchain client* to handle bi-directional communication with the on-chain aggregator. The local model updates are transformed into blockchain transactions and forwarded to the on-chain aggregator. In turn, the blockchain client reads global model updates from the on-chain aggregator and provides them to the zk-trainer through the middleware. The *middleware* manages inputs to and outputs of the zk-trainer.

The *zk-trainer* implements a learning algorithm as a zero-knowledge computation to train the local model using both, inputs from the data source and the iteratively updated global model. In addition to the updated local model, the zk-trainer produces a ZKP that attests to the computational correctness of the local training processes. To create the ZKP, the zk-trainer uses a dedicated proving key that is part of an asymmetric key pair. ZKPs are verified using the corresponding verification key which is part of the on-chain aggregator.

2) *On-chain Aggregator*: The on-chain aggregator is implemented as smart contracts that are redundantly executed on the blockchain infrastructure. It consists of two logical components: one for verifying ZKPs and one for managing the global model.

The *ZKP-verifier* verifies ZKPs attached to local model updates using the verification key that is the other part of the asymmetric key pair. Thereby, it validates that the updates have been computed correctly. Given that local off-chain computations are the same on all learning nodes, only one proving-verification key pair is applied per setup.

The *global model manager* aggregates the verified local model updates into the global model. Serving as a single

endpoint, the on-chain aggregator receives these updates from all participating learning nodes and, thereby, maintains a single source of truth throughout the nodes of the blockchain network. As a central challenge, the on-chain aggregator must, thereby, guarantee *fairness*, i.e., only one update is applied per node in each cycle, and *liveness*, i.e., a new cycle is initiated eventually. Therefore, the on-chain aggregator keeps a list of all participants who we assume to be pre-registered. Addressing fairness, the on-chain aggregator checks that the learning nodes only submit one update per cycle at max. To guarantee liveness, the on-chain aggregator initiates a new cycle after a predefined period, determined based on the block-based timestamp.

B. Workflow

As depicted in Figure 2, the system workflow comprises an initial one-time *setup* and repeated *updating cycles*. In the workflow, four parties are involved: the developer who is exclusively involved in the setup phase, the on-chain aggregator, the zk-trainer, and the data source. For simplicity, in Figure 2, the on-chain aggregator is represented as a single component, and the node manager is omitted since it transmits data only.

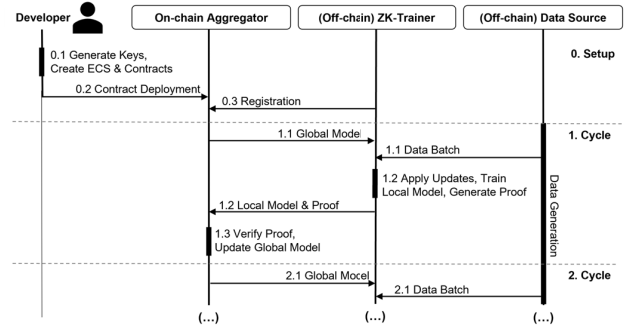


Fig. 2: Workflow

Setup: During the one-time setup, the developer creates the zk-trainer as the off-chain prover and the on-chain aggregator as the on-chain verifier. In this paper, we assume that this VOC instantiation is realized with ZoKrates and, hence, follows the setup flow described in [18]: The developer defines the training algorithm in the ZoKrates high-level language, compiles it into an executable constraint system (ECS), and, based on the ECS, generates the proving-verification key pair. The zk-trainer requires the proving key and the ECS, whereas the on-chain aggregator holds the verification key and implements the ZKP verification logic. The implemented on-chain aggregator contracts for global model management and ZKP verification are then deployed to the blockchain. Once deployed, learning nodes can be registered with their blockchain account address for participation.

Updating Cycles: At the beginning of a cycle, the zk-trainer reads the latest global model from the on-chain aggregator and applies the contained parameters, e.g., weights and biases, to the local model. It receives the inputs for the training as a data

batch of fixed size and data format from the data source. Based on the global model parameters and the input data, the zk-trainer executes the training as a zero-knowledge computation. Therefore, it executes the ECS which returns a witness, i.e., an input-specific variable assignment of the ECS. Based on the witness and the proving key, it generates the ZKP which, together with the updated local model, it, then, sends to the on-chain aggregator as a blockchain transaction. On receiving the transaction, the smart contract first verifies the proof with the verification key and, if successful, updates the global model with the received local model parameters.

IV. IMPLEMENTATION

Given the generality of VOC [24], the proposed system architecture is applicable to various federated learning models. Without mitigating this generality, in this section, we demonstrate technical feasibility of our proposal by implementing a simple *feedforward neural network* [20] with one hidden layer (compare Figure 3 and our proof of concept (PoC) implementation²).

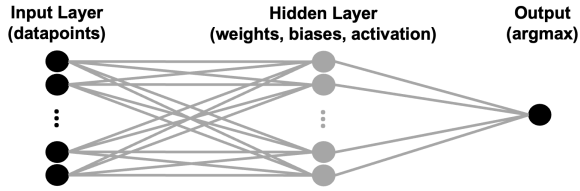


Fig. 3: Feedforward Neural Network with One Hidden Layer

A. Off-chain Learning Node

In our PoC, the zk-trainer and the node manager represent the core components of the off-chain learning node.

1) *ZK-Trainer*: As most distinguishing aspect of our PoC, the depicted feedforward neural network is implemented using the ZoKrates³ high-level language and executed as a zkSNARKs-based computation. For this, we leverage ZoKrates CLI commands with their current default settings, i.e., the Groth16 proving scheme [25] and the alt_bn128 (Barreto-Naehrig) curve.

The program takes the current batch of input data as private inputs and the learning rate and latest global model as public inputs. Based on these arguments, it executes the matrix calculations on the hidden layer and applies the argmax function to determine the prediction. The loss function is implemented as the mean squared error function and its derivative. Return values are the ZKP as well as the new weights and biases representing the updated local model.

While providing Turing-completeness, the ZoKrates programming language comes with some constraints that impact our implementation of the learning algorithm, noteworthily with regards to two aspects: First, in ZoKrates only unsigned

integers can be used as data types. Consequently, float numbers contained in the input data are scaled to enable integer-based operations. Furthermore, for every value, a corresponding Boolean is passed to the program which indicates whether the value is positive or negative. Arithmetic operations are rewritten to account for possible over- and underflows and return the signs of the resulting values.

Second, in ZoKrates, the size and the type of the input arguments must be determined at compile time. Consequently, the number of datapoints of the input layer cannot be dynamically changed. This also impacts the implementation of exponential activation functions where exponents are dynamically calculated based on the input data. Hence, in our PoC implementation, only linear relationships between the features and truth labels are predicted.

2) *Node Manager*: The node manager's *message broker* is implemented with RabbitMQ⁴, a broker that enables simple communication with low power consumption, suitable for IoT settings. It supports multiple data sources and, for each, provides a dedicated message queue. This does not only enable off-chain learning nodes with multiple data sources but also helps to realize the testbed for our experiments described in Section VI.

Published messages are consumed by the Python-based *middleware* which executes instances of the zk-trainer by calling the ZoKrates CLI: First, the witness is computed based on the ECS with the `compute-witness` command which takes all input arguments. Second, the proof is generated with the `generate-proof` command which takes the witness and the proving key as arguments. The intermediate witness file is written to and read from the local file system.

Outputs are provided to the on-chain aggregator through the *blockchain client* which relies on the web3.py Python library⁵ to communicate with the on-chain aggregator. It uses the blockchain account address of the local learning node to transmit the ZKP and the local model as a blockchain transaction to the on-chain aggregator.

B. On-chain Aggregator

The on-chain aggregator is implemented in Solidity⁶ and separates the functionality into two Ethereum [22] smart contracts: the learning and the verifier contract.

1) *Learning Contract*: The learning contract provides the updating mechanism for the global model and, as the common endpoint for the off-chain learning nodes, implements the management of the updating cycles introduced in Section III. Analogue to the local model, the global model consists of the weight-vector and the bias-matrix. The updating mechanism requires the smart contract to store two versions: One that is only updated at the end of the cycle and is consumable by off-chain learning nodes. And a temporary one that is used to aggregate the updated local models of the participating learning nodes using a simple moving average function. At

²<https://github.com/NikolasHaimerl/Advancing-Blockchain-Based-Federated-Learning-Through-Verifiable-Off-Chain-Computations>

³<https://github.com/Zokrates/ZoKrates>

⁴<https://www.rabbitmq.com/>

⁵<https://github.com/ethereum/web3.py>

⁶<https://github.com/ethereum/solidity>

the end of an updating cycle, the temporary model replaces the other.

2) *Verifier Contract*: The verifier contract is called from the learning contract before an update is applied to the global model. It implements the ZKP verification logic in the *verifyTx()* function and treats the verification key as hard-coded parameter. The contract is automatically generated as a Solidity smart contract by means of the ZoKrates CLI.

V. APPLICATION

Given a technical realization of our architecture, in this section, we introduce an application context where the model's integrity and, hence, the verifiability of both, the local training and the global model management, are of particular relevance. Based on that, we identify and describe a suitable dataset that serves as a basis for our evaluation.

A. Context

Remote, *in-home health tracking* through wearables is of paramount importance if an individual's health condition restricts its mobility such that routine examinations at the doctor or hospitals cannot be fulfilled anymore [26], [27]. In such contexts, "smart" or learning-based health applications that monitor an individual's health condition through wearables and predict worsening health conditions can help saving lives, e.g., by informing healthcare services or relatives on time. Such applications often provide classification problems, e.g., predicting health states (good, fair, serious, critical) from measurements collected by wearables.

Federated learning has been applied for in-home health monitoring to improve the learning models of such smart health applications through a joint training of a global learning model by connected households [26], [27]. Since personal health data is highly confidential and requires special protection as required by privacy regulations such as the EU's GDPR⁷ or the United States' HIPAA⁸, federated learning can help comply with these regulations and keep this data private. Furthermore, the integrity of the global learning model is of utmost importance since false predictions of the health condition can have dramatic consequences, e.g., lead to death. Therefore, guarantees of global verifiability and tamper resistance have high priority in such application contexts. This makes our system proposal particularly suitable for such a scenario.

If an in-home health monitoring system as proposed in [26] or [27] is realized with our system architecture, each *household* represents a learning node. The data source is represented by a *wearable*, e.g., a smartwatch, which is worn by the patient and continuously generates health data. The smartwatch transmits the data to an in-home *workstation* which hosts all other components of the learning node and the on-chain aggregator's smart contracts. Therefore, it also provides the networking interface to other connected household and healthcare services.

⁷<https://gdpr.eu>

⁸<https://www.hhs.gov/hipaa/for-professionals/privacy/index.html>

B. Dataset

To represent the described use case, we select a public dataset that is closely related to the described use case in that it is built upon sensor data generated from wearables and provides a classification problem. The dataset is available under the UCI Repository for Daily and Sports Activities Data Set⁹ and has been generated by eight subjects (four females and four males between the ages 20 and 30) who executed activities for a duration of five minutes each. To adopt the dataset to our use case and make it suitable for our proposed system, we apply two major changes:

First, we reduce the feature space. In total, the dataset comprises 45 features which originate from measurements in different spatial directions generated by different sensor devices attached to different body parts. To adopt the dataset to our use case where a single smartwatch is used as data source, we reduce the dataset to measurements of one sensor device resulting in nine features which determine the size of the neural network's input vector.

Second, we reduce the set of prediction classes. The dataset comprises 19 different activities, i.e., prediction classes. To reduce complexity and align the data with our use case and learning model, we combined activities that characterize through similar measurements, e.g., ascending and descending stairs, or exercising in a stepper and on a cross-trainer. This results in a total of six prediction classes which determine the size of the hidden layer's biases vector and the weight matrix.

VI. EVALUATION

To evaluate the system, we deploy the PoC implementation in a test environment and analyze its behavior during experimentation. Therefore, we describe our experimental setup and discuss results obtained from the experiments with regard to computational and learning performance.

A. Experimental Setup

Aligned with our use case, we spawn eight in-home health monitoring systems that act as off-chain learning nodes and are managed by a smart contract-based on-chain aggregator. The on-chain aggregator runs on a virtual blockchain network which is instantiated with Ganache¹⁰. Each household is represented by one data source, one ZoKrates-based zk-trainer, and one blockchain account. To simulate the different data sources, we implement a Python-based *workload generator* that retrieves the dataset described in Section V-B from the filesystem and splits it into eight subsets each representing a data source. From the subsets, the workload generator publishes data batches from each data source to the corresponding queue provided by the message broker.

Given that our node manager implementation is capable of managing multiple data sources and multiple zk-trainers, it is not replicated for each household but instead, in our experimental setup, manages multiple household instances.

⁹ <https://archive.ics.uci.edu/ml/datasets/daily+and+sports+activities#>

¹⁰<https://github.com/trufflesuite/ganache>

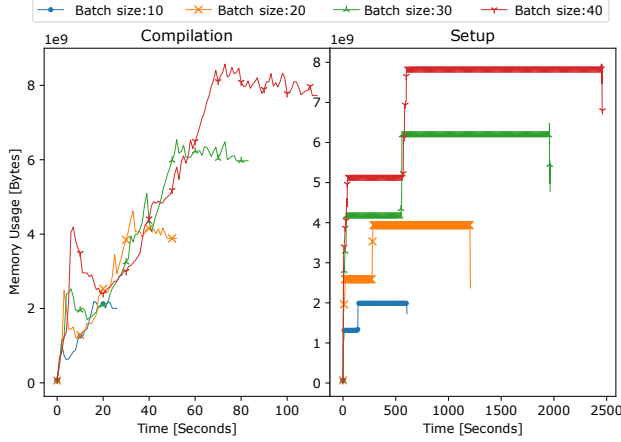


Fig. 4: Memory requirements of compilation and setup with different batch sizes.

As such, it employs eight zk-trainers which run in parallel during a cycle. The resulting local models are provided to the on-chain aggregator through the Ganache blockchain client which provides the models of each household through the corresponding account address.

To simulate this environment and conduct the experiments, we use a single machine equipped with an AMD Ryzen 3950X@3.5 GHz CPU with 16 cores, memory capacity of 128 GB@3600 MHz (DDR4), and a solid-state-drive with approximately 3500 MB/s read and write capabilities. The experiments are executed three times and the results represent the average of the measurements gathered in each run. In addition, we show the standard deviations, where meaningful.

B. Computational Costs

To gain insights into the computational performance of our system, we observe off- and on-chain computations during the one-time setup and the updating cycles discussed in Section III. We measure the execution time [sec] and memory consumption [Bytes] for off-chain computations and transaction costs [Gas¹¹] for on-chain computation.

To understand the system behavior for varying workloads, we conducted the experiments with four different batch sizes comprising 10/20/30/40 datapoints respectively per cycle (cf. Table I).

The *setup phase* comprises two off-chain computations: the compilation of the ZoKrates program and the key generation (in Figure 4 referred to as setup following the ZoKrates command name), and the on-chain aggregator deployment as on-chain computations. As depicted in Figure 4, execution time and memory consumption behave rather linearly to the increasing batch sizes. While during the key generation (setup) the memory consumption is rather static, it grows over the execution time during compilation. The latter may be caused by an increasing memory space allocated for the constraint

¹¹Gas is an Ethereum-specific metric for measuring transaction complexity.

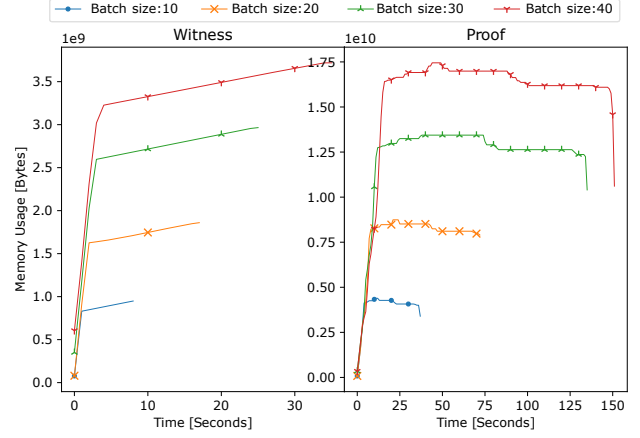


Fig. 5: Memory requirements of computing witnesses and the proof with different batch sizes.

TABLE I: Average execution time and memory consumption for varying batch sizes.

Batch size [#]	10	20	30	40
Compile Time [sec]	27.79	56.55	93.01	127.46
STD [sec]	0.02	0.87	3.06	5.70
Compile Memory [GB]	2.26	4.62	6.82	9.31
STD [MB]	80.5	18.9	18.0	56.3
Setup Time [sec]	651.35	1290.40	2107.88	2711.53
STD [sec]	2.00	1.20	7.33	56.87
Setup Memory [GB]	2.02	4.12	6.68	8.28
STD [MB]	23.82	0.00	157.51	114.405
Witness Time [sec]	8.90	19.21	27.43	37.76
STD [sec]	0.50	1.50	0.50	0.50
Witness Memory [GB]	0.95	1.88	2.96	3.72
STD [MB]	2.45	0.00	0.00	0.00
Proof Time [sec]	39.55	78.25	145.50	167.56
STD [sec]	0.03	1.08	1.05	4.34
Proof Memory [GB]	4.39	8.74	13.43	17.45
STD [MB]	0.00	0.00	0.00	0.00

system which is built during compilation. The initial one-time deployment cost of the on-chain aggregator comprises 0.00624 Gas for the *ZKP-verifier* and 0.0023 Gas for the *global model manager* smart contracts.

During the *operational phase*, in each cycle the witness generation and the proof construction are executed as off-chain computations whereas proof verification and the model update are executed on-chain. As depicted in Figure 5, memory size and execution time grows rather linearly with increasing batch sizes. Clearly, proof construction is more expensive than witness generation. The transaction costs for verification range between approximately 0.0024 Gas (batch size: 10) and 0.0025 Gas (batch size: 40) per cycle. However, compared to the one-time setup, repeating computations in the operational phase require considerably fewer resources, take less time, and are therefore evaluated as practically feasible.

C. Federated Learning Performance

In addition to the computational performance, insights into the performance of the applied learning model are essential to

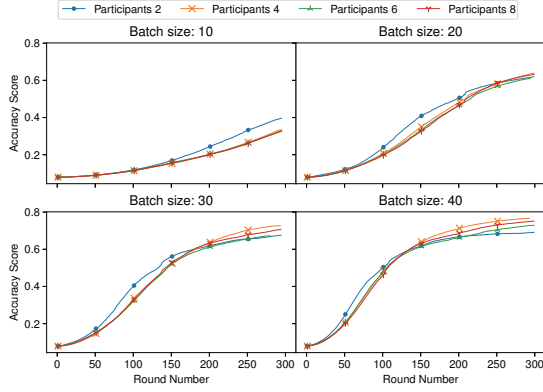


Fig. 6: Accuracy scores for different numbers of participants and various batch sizes.

evaluate its applicability. Therefore, we measure the model's accuracy and how it behaves with a varying number of participating off-chain learning nodes (2, 4, 6, 8). Again, experiments are conducted with the four different batch sizes (10, 20, 30, 40) with a total number of 300 training cycles each. Different input data batches are also randomly chosen from the data set to avoid over-fitting of the learning model. Results are depicted in Figure 6.

As expected, the more datapoints are used per cycle, the faster the model's output gains accuracy. While the improvement curves for batch sizes 20, 30 and 40 all have their inflection point within 300 updating rounds, this is not yet the case for batch size 10. With higher batch sizes, better accuracy is reached even more early. Our model converges towards a 0.79 accuracy score with batch size 40, which is considered a very satisfactory result with respect to the simple feed-forward neural network applied in our proof-of-concept architecture due to the limitations imposed by the ZoKrates language. With a more sophisticated model – which itself was deliberately not subject of our optimizations – the accuracy is expected to be even higher.

Moreover, while all runs seem to converge towards the same maximum accuracy performance eventually, a discrepancy is noticeable between the number of participants involved. Contrary to what was first expected, the test runs with only 2 learning nodes started off better than the tests with 4, 6 or 8 participating nodes in all variations of the batch size. After investigation, this was caused by the diversity of the data assigned to the nodes. Naturally, some nodes are better suited to train a global model since their data are by chance closer to the general median than others. Therefore, we consider this a minor irregularity in the dataset which would not occur in real-world use cases with more participants.

Lastly, with more participants, a diminishing performance improvement can be observed. While more participants may result in a slower start of the training improvement, the convergence accuracy score is the highest with the most participants. This is to be expected as the experiment's data

were sampled from all nodes.

VII. RELATED WORK

In this section, we discuss related work around the two attack vectors that we mitigate with our system proposal, i.e., the local model training and the global model management.

Addressing limitations of a centralized architecture as described in Section I, more decentralized setups have been proposed, with blockchains representing one possible technological enabler [8]–[13]. As revealed in our evaluation, a blockchain-based setup, however, also introduces undesirable overheads that can quickly stress resource capacities in constrained environments. In this regard, literature provides approaches for more efficient federated learning implementations as general approaches, e.g., [8], or simply by selecting permissioned blockchains that implement more lightweight consensus protocols by design, e.g., [28]. While our PoC has not been optimized for storage and computation, the proposed concept is blockchain agnostic which allows the developer to decide on a deployment with respect to use case-specific requirements.

The other attack vector is exposed by the local training which, especially in distributed systems, can leverage for poisoning attacks by malicious learning nodes [6], [7]. Addressing this attack in blockchain-based architectures, incentive schemes have been proposed that reward honestly acting nodes while penalizing dishonest nodes in turn [14]–[17]. Compared to our approach, incentive schemes rely on stronger assumptions and highly rely on cryptocurrency-enabled blockchains which restricts their applicability.

As another approach, the local training and the global model management can be executed by Trusted Execution Environments (TEE) which guarantee for computational correctness and even enable non-revealing processing of confidential data [29]–[33]. TEEs provide hardware-level protection against external access to data in use, i.e., during runtime, and allow third parties to verify the authenticity of messages using hardware-level asymmetric encryption. While TEEs represent an alternative to ZKPs to realize off-chain computations as described in [24], they have not yet been applied for VOC-enabled federated learning. Compared to ZKPs, TEEs have weaker security guarantees and rely on a trusted hardware manufacturer. However, they provide better performance and have a smaller computational overhead as shown in [34].

VIII. CONCLUSION

We presented a model and first system architecture for blockchain-based federated learning with verifiable off-chain computations. We implemented the architecture based on a ZoKrates-based feedforward network, instantiated it in a smart healthcare scenario, and evaluated the implementation and findings also regarding learning performance and computational costs. We can conclude that on-chain verifiability of correctness of off-chain learning processes not only is feasible in specific deployments and learning scenarios, but presents an

important if not significant direction to federated learning and privacy-enhanced decentralized applications in general.

In our future work, we want to seize on aspects revealed in Section VI by further investigating mechanisms to decrease the costs for verifying federated learning results, e.g., by off-chaining parts of the necessary storage, and to remove yet existing trust assumptions of the learning nodes, e.g., through trustworthy pre-processing [34]. In addition, we want to take a look at approaches to scale the solution to handle larger problem sizes.

ACKNOWLEDGMENT

The financial support by the Austrian Federal Ministry for Digital and Economic Affairs, the National Foundation for Research, Technology and Development as well as the Christian Doppler Research Association is gratefully acknowledged.

REFERENCES

- [1] Z. Ullah, F. Al-Turjman, L. Mostarda, and R. Gagliardi, "Applications of Artificial Intelligence and Machine learning in smart cities," *Computer Communications*, vol. 154, pp. 313–323, 2020.
- [2] W. Li, Y. Chai, F. Khan, S. R. U. Jan, S. Verma, V. G. Menon, Kavita, and X. Li, "A Comprehensive Survey on Machine Learning-Based Big Data Analytics for IoT-Enabled Smart Healthcare System," *Mobile Networks and Applications*, vol. 26, no. 1, pp. 234–252, 2021.
- [3] T. S. Brisimi, R. Chen, T. Mela, A. Olshevsky, I. C. Paschalidis, and W. Shi, "Federated learning of predictive models from federated electronic health records," *International Journal of Medical Informatics*, vol. 112, pp. 59–67, 2018.
- [4] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konečný, S. Mazzocchi, H. B. McMahan, T. V. Overveldt, D. Petrou, D. Ramage, and J. Roselander, "Towards federated learning at scale: System design," *CoRR*, vol. abs/1902.01046, 2019. [Online]. Available: <http://arxiv.org/abs/1902.01046>
- [5] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated learning: Challenges, methods, and future directions," *IEEE Signal Processing Magazine*, vol. 37, no. 3, pp. 50–60, 2020.
- [6] M. Fang, X. Cao, J. Jia, and N. Gong, "Local Model Poisoning Attacks to Byzantine-Robust Federated Learning," in *29th USENIX Security Symposium*. USENIX Association, 2020, pp. 1623–1640.
- [7] C. Ma, J. Li, M. Ding, H. H. Yang, F. Shu, T. Q. S. Quek, and H. V. Poor, "On Safeguarding Privacy and Security in the Framework of Federated Learning," *IEEE Network*, vol. 34, pp. 242–248, 2020.
- [8] Y. Li, C. Chen, N. Liu, H. Huang, Z. Zheng, and Q. Yan, "A blockchain-based decentralized learning framework with committee consensus," *IEEE Network*, vol. 35, pp. 234–241, 2020.
- [9] Y. Lu, X. Huang, Y. Dai, S. Maharjan, and Y. Zhang, "Blockchain and learning for privacy-preserved data sharing in industrial IoT," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 6, p. 4177–4186, 2020.
- [10] Y. J. Kim and C. S. Hong, "Blockchain-based node-aware dynamic weighting methods for improving federated learning performance," in *2019 20th Asia-Pacific Network Operations and Management Symposium (APNOMS)*. IEEE, 2019.
- [11] J. Li, Y. Shao, K. Wei, M. Ding, C. Ma, L. Shi, Z. Han, and V. Poor, "Blockchain assisted decentralized federated learning (BLADE-FL): Performance analysis and resource allocation," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, pp. 2401–2415, 2022.
- [12] S. K. Lo, Y. Liu, Q. Lu, C. Wang, X. Xu, H. Paik, and L. Zhu, "Blockchain-based trustworthy federated learning architecture," *CoRR*, vol. abs/2108.06912, 2021. [Online]. Available: <https://arxiv.org/abs/2108.06912>
- [13] K. Toyoda, J. Zhao, A. Zhang, and P. T. Mathiopoulos, "Blockchain-enabled federated learning with mechanism design," *IEEE Access*, vol. 8, pp. 219 744–219 756, 12 2020.
- [14] H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Blockchained on-device federated learning," *IEEE Communications Letters*, vol. 24, pp. 1279–1283, 2019.
- [15] J. Weng, J. Weng, J. Zhang, M. Li, Y. Zhang, and W. Luo, "Deepchain: Auditable and privacy-preserving deep learning with blockchain-based incentive," *IEEE Transactions on Dependable and Secure Computing*, vol. 18, no. 5, pp. 2438–2455, 2021.
- [16] X. Bao, C. Su, Y. Xiong, W. Huang, and Y. Hu, "FLChain: A blockchain for auditable federated learning with trust and incentive," in *2019 5th International Conference on Big Data Computing and Communications*. IEEE, 2019, pp. 151–159.
- [17] L. Feng, Z. Yang, S. Guo, X. Qiu, W. Li, and P. Yu, "Two-layered blockchain architecture for federated learning over mobile edge network," *IEEE Network*, pp. 1–14, 2021.
- [18] J. Eberhardt and S. Tai, "ZoKrates – scalable privacy-preserving off-chain computations," *2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, pp. 1084–1091, 2018.
- [19] J. Konečný, H. B. McMahan, F. X. Yu, P. Richtárik, A. T. Suresh, and D. Bacon, "Federated learning: Strategies for improving communication efficiency," *CoRR*, vol. abs/1610.05492, 2016. [Online]. Available: <http://arxiv.org/abs/1610.05492>
- [20] G. Bebis and M. Georgiopoulos, "Feed-forward neural networks," *IEEE Potentials*, vol. 13, no. 4, pp. 27–31, 1994.
- [21] Q. Yang, Y. Liu, T. Chen, and Y. Tong, "Federated machine learning: Concept and applications," *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019.
- [22] G. Wood et al., "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Project Yellow Paper*, 2014.
- [23] M. Castro and B. Liskov, "Practical byzantine fault tolerance and proactive recovery," *ACM Transactions on Computer Systems*, vol. 20, no. 4, pp. 398–461, 2002.
- [24] J. Eberhardt and J. Heiss, "Off-chaining Models and Approaches to Off-chain Computations," in *2nd Workshop on Scalable and Resilient Infrastructures for Distributed Ledgers*. ACM, 2018, pp. 7–12.
- [25] J. Groth, "On the Size of Pairing-Based Non-Interactive Arguments," in *35th Annual International Conference on Advances in Cryptology*, ser. LNCS, vol. 9666. Springer, 2016, p. 305–326.
- [26] Q. Wu, X. Chen, Z. Zhou, and J. Zhang, "FedHome: Cloud-Edge based Personalized Federated Learning for In-Home Health Monitoring," *IEEE Transactions on Mobile Computing*, 2020.
- [27] D. Gupta, O. Kayode, S. Bhatt, M. Gupta, and A. Tosun, "Hierarchical Federated Learning based Anomaly Detection using Digital Twins for Smart Healthcare," in *2021 IEEE 7th International Conference on Collaboration and Internet Computing*. IEEE, 2021, pp. 16–25.
- [28] V. Mothukuri, R. M. Parizi, S. Pouriyeh, A. Dehghantaha, and K.-K. R. Choo, "FabricFL: Blockchain-in-the-loop federated learning for trusted decentralized systems," *IEEE Systems Journal*, 2021.
- [29] O. Ohrimenko, F. Schuster, C. Fournet, A. Mehta, S. Nowozin, K. Vaswani, and M. Costa, "Oblivious multi-party machine learning on trusted processors," in *25th USENIX Security Symposium*. USENIX Association, 2016, pp. 619–636.
- [30] N. Hynes, R. Cheng, and D. Song, "Efficient deep learning on multi-source private data," *CoRR*, vol. abs/1807.06689, 2018. [Online]. Available: <http://arxiv.org/abs/1807.06689>
- [31] J. G. Chamani and D. Papadopoulos, "Mitigating leakage in federated learning with trusted hardware," *CoRR*, vol. abs/2011.04948, 2020. [Online]. Available: <https://arxiv.org/abs/2011.04948>
- [32] Y. Chen, F. Luo, T. Li, T. Xiang, Z. Liu, and J. Li, "A training-integrity privacy-preserving federated learning scheme with trusted execution environment," *Information Sciences*, vol. 522, pp. 69–79, 2020.
- [33] F. Mo, H. Haddadi, K. Katevas, E. Marin, D. Perino, and N. Kourtellis, "PPFL: Privacy-preserving federated learning with trusted execution environments," in *19th Annual International Conference on Mobile Systems, Applications, and Services*. ACM, 2021, pp. 94–108.
- [34] J. Heiss, A. Busse, and S. Tai, "Trustworthy Pre-Processing of Sensor Data in Data On-chaining Workflows for Blockchain-based IoT Applications," in *19th International Conference on Service-Oriented Computing*, ser. LNCS, vol. 13121. Springer, 2021, pp. 627–640.